

Контракт «приложение ↔ client-сервер» — черновик v0

Дата: 2026-08-20 · Заменяет инвентарную часть [wire-surface.md](#). Сверен с приложением: каждая команда и каждое поле проверены против реального кода `lib/`.

Этапность реализации:

Этап	Что	Где
1 — инфраструктура, без аутентификации	Конверт §2, WebSocket-команды §3–§6, REST-поверхность §1/§7, SQLite	Реализация — каталог <code>client_backend/</code> этого репозитория
2 — безопасность	Спаривание и аутентификация по authentication.md (документ одобрен; сознательно не на первом этапе) — §8.1	там же

Конверт помечен **v0**: пока обе стороны наши, правка формата — дешёвая; поле `schema` в рукопожатии существует ровно для этого.

1. Транспорт

Постоянный **WebSocket поверх TLS** (`wss`), порт 443, пиннинг ключа сервера из ссылки спаривания, самоподписанный сертификат. Внутри кадров — **JSON**. Само спаривание (откуда берутся адрес, пиннутый ключ и токен) — вне этого контракта: [../architecture/authentication.md](#).

REST-поверхность этапа 1 (тот же процесс, тот же порт):

Endpoint	Что
<code>PUT /files/{upload_token}</code>	байты вложения (§7)
<code>GET /files/{download_token}</code>	скачивание с поддержкой Range (§7)
<code>GET /health</code>	живость процесса — для служб ОС и туннеля
локальная страница спаривания	только LAN-интерфейс; вне контракта (authentication.md)

Формат времени: все поля `*_at` — целое unix-время в секундах, UTC. Других форматов дат в протоколе нет.

2. Конверт кадра — v0

Четыре вида кадров. Других нет.

Приветствие сервера (сервер → приложение, ровно один раз при открытии сокета, до любых команд):

```
{"srv": {"schema_max": 1, "challenge": "<base64, 32 байта>"}}
```

`challenge` подписывается ключом устройства в `session.hello` (§3); на этапе 1 поле присутствует, но подпись не проверяется.

Команда (приложение → сервер):

```
{"id": 7, "cmd": "message.send", "data": { ... }}
```

Ответ на команду (сервер → приложение):

```
{"id": 7, "ok": true, "data": { ... }}
```

```
{"id": 7, "ok": false, "error": {"code": "name_taken", "message": "Chat name already ex:"}}
```

Событие (сервер → приложение):

```
{"seq": 1042, "event": "message.new", "data": { ... }}
```

Правила:

- `id` — корреляция команда↔ответ: выдаёт клиент, монотонно в рамках соединения; каждый ответ несёт `id` своей команды. Сопоставление устаревших ответов — только по `id`, никогда по содержимому.
- `seq` — сквозная нумерация событий: выдаёт сервер, **строго возрастающая на всю базу** (единственный писатель + AUTOINCREMENT). `seq` — это и есть курсор синхронизации.
- Один кадр — один JSON-объект; вид кадра различается по наличию `srv` / `cmd` / `ok` / `event`.

2.1. Коды ошибок v0

Код	Когда	Реакция клиента и повторяемость
<code>invalid_request</code>	Кадр не разобран, нет обязательного поля, нарушены правила поля (длина, пустота)	Ошибка программиста — лог, фатальный экран; не повторять
<code>not_found</code>	Сущность по id не существует	Контекстно; не повторять
<code>name_taken</code>	Имя чата / label занято	Inline у поля ввода; не повторять
<code>payload_too_large</code>	Превышен лимит из рукопожатия	Inline в композере; не повторять — предотвращается предполётной проверкой лимитов
<code>attachment_gone</code>	Байты вложения вычищены по TTL	Терминальное error-состояние экрана файла (5.3), без кнопки повтора — не фатальный экран всего приложения
<code>rate_limited</code>	Слишком часто	Автоповтор с backoff
<code>internal</code>	Ошибка сервера	Inline-повтор вручную
<code>unauthenticated</code>	Подпись/сессия не прошли	§8.1; на этапе 1 не используется
<code>unsupported_schema</code>	Версия контракта клиента не поддерживается	Фатальный экран с предложением обновить приложение

Правило эволюции: **неизвестный клиенту код трактуется как `internal`** (inline-повтор). Коды спаривания (`invalid_token` , `token_expired`) войдут в таблицу на этапе 2.

3. Рукопожатие и синхронизация

Первая команда после открытия сокета — всегда `session.hello` (единственное будущее исключение — `pair` неспаренного устройства, §8.1):

```
{
  "id": 1, "cmd": "session.hello", "data": {
    "schema": 1,
    "since": 1041,
    "device_key": "<base64 Ed25519 pub>",
    "signature": "<base64: подпись challenge>"
  }
}
```

```
{
  "id": 1, "ok": true, "data": {
    "schema": 1,
    "cursor": 1042,
    "limits": {
      "max_message_bytes": 65536,
      "max_attachment_bytes": 104857600,
      "max_frame_l
    },
    "identity": {
      "id": "<формат - Q11>",
      "label": "User7421"
    }
  }
}
```

- `schema` — версия контракта; несовместимость → `unsupported_schema`.
- `since` — курсор клиента: последний обработанный `seq`. Сервер после ответа досылает **все события с `since+1`** и переходит к живому потоку. Порядок строго: **подписка** → **replay** → **live**; дубликаты на стыке допустимы (клиент дедуплицирует по `seq`), потеря — нет.
- **Первое подключение устройства:** `since` опускается — `replay` не выполняется; клиент бутстрапится командами `chats.list / messages.list` и **принимает `cursor` из ответа как стартовый**.
- **Правило «догнан»:** клиент считается догнанным, когда обработан `seq ≥ cursor` из ответа `hello` (при `since == cursor` — сразу). Это единственный способ отличить конец `replay` от `live` — сами события неотличимы.
- `limits` — объявленные сервером пределы; клиент обязан проверять их **до** отправки (ограничитель в композере, предполётная проверка размера вложения).
- `identity` — авторитет по собственной личности: `id` — собственный `author-id` клиента (им ключуется `own/other`-разметка сообщений; формат — за Q11), `label` — текущее имя. **Обязанность клиента:** персистить оба в сессию при каждом подключении и раскатывать по поверхностям (существующий канал `watchLabel`); префилл экрана выбора имени берётся отсюда, не из константы. Это же закрывает догоняющее обновление `label`, изменённого с другого устройства офлайн.
- `device_key / signature` — аутентификация подключением: подпись `challenge` из приветствия сервера (§2) ключом устройства. Входа по идентификатору-секрету в контракте **не существует** — устройство авторизуется спариванием, соединение — подписью. ⚠ **На этапе 1 поля игнорируются** — это единственное отличие этапа 1 от контракта.

Принцип, стоящий за `since / seq`: уведомление — оптимизация, а не транспорт; единственный гарантированный момент синхронизации — когда приложение само вышло на связь. Догоняющая синхронизация обязательна при каждом появлении на переднем плане, реконнекте и пробуждении машины — поэтому сервер отдаёт «что произошло с курсора X», а не «последние N».

4. Команды: чаты — этап 1

Команда	data запроса	data ответа
<code>chats.list</code>	<code>{page, page_size, query?}</code>	<code>{chats: [Chat], has_more}</code>
<code>chat.get</code>	<code>{chat_id}</code>	<code>{chat: Chat}</code>
<code>chat.create</code>	<code>{name}</code>	<code>{chat: Chat}</code> — сервер авторитет по уникальности имени
<code>chat.rename</code>	<code>{chat_id, name}</code>	<code>{chat: Chat}</code> — переименовать может любой (открытое пространство); проверка уникальности исключает сам <code>chat_id</code>
<code>chat.nameAvailable</code>	<code>{name, exclude_chat_id?}</code>	<code>{available: bool}</code>
<code>chat.files</code>	<code>{chat_id, before_seq?, limit}</code>	<code>{files: [Attachment + message_id + seq], has_more}</code> — полный список вложений чата независимо от глубины локальной истории (панель 5.4)

Правила поля `name`: `trim` по краям; непустое после трима; **≤64 символов**; `charset` неограничен; нарушение → `invalid_request`. **Уникальность имени чата — регистронезависимая** (правило регистрозависимости относится к `label`, не к имени чата): `chat.create` / `chat.rename` / `chat.nameAvailable` применяют одно правило при выдаче `name_taken` / `available:false`.

Пагинация: `page` — 1-базный; `page_size` выбирает клиент (серверный потолок 100). Порядок — по `last_activity_at` по убыванию; порядок живой, поэтому клиент **дедуплицирует строки по `chat_id`** при склейке страниц. `query` — регистронезависимое вхождение подстроки в `name`, `trim` по краям; пустая строка = нет фильтра.

`chat.get` обязателен: входящее событие о чате **за пределами загруженного префикса** иначе ссылается на недостижимую сущность (пуш по чату из холодного старта, восстановление выбора на десктопе — те же случаи).

Chat (wire):

```
{"chat_id": "c_9f2", "name": "Kitchen talks",
 "created_at": 1755600000, "created_by_label": "Anna",
 "last_message_preview": "...", "last_activity_at": 1755600123}
```

- `created_by_label` — `label` создателя: из него + `created_at` клиент рендерит генезис-строку треда («Chat created by ...»). **Системных сообщений на проводе нет** — генезис-строка всегда собирается клиентом.

- `last_activity_at` изменяют **только сообщения и создание чата**; `chat.rename` его не трогает — строка не прыгает в списке.
- Превью и порядок — авторитет сервера **для страниц** `chats.list`; при живом событии `message.new` превью/порядок обновляет клиент из данных события (§6).

⚠ Поля `unread_count` в wire-модели **нет** — счётчик устройство-локальный (§8.3). ⚠
 При «E2EE на устройстве» (Q1) серверное превью и `chat.files` невозможны в открытом виде — тот же шов, что у `body` (§5).

5. Команды: сообщения — этап 1

Команда	data запроса	data ответа
<code>messages.list</code>	<code>{chat_id, before_seq?, limit}</code>	<code>{messages: [Message], has_more}</code> — обратная пагинация; <code>total</code> не существует
<code>message.send</code>	<code>{chat_id, client_message_id, body?, attachment?: {file_id}}</code>	<code>{message: Message}</code> — серверное эхо

- `body` опционален при наличии `attachment` (сообщение-вложение без текста — штатный случай); хотя бы одно из двух обязательно.
- **Авторства в запросе нет:** `author_id` / `author_label` сервер берёт из сессии и денормализует сам.
- `client_message_id` (UUID) — обязателен: ключ идемпотентности. Сервер хранит его и при повторе возвращает прежнее эхо. **Эхо и `message.new` собственных сообщений несут `client_message_id`** — очередь исходящих сверяется с replay после рестарта без холостого resend.
- **Таймаут отправки — 10 секунд** (согласован с keepalive §9): нет ответа → локальный статус `error`, повтор с тем же `client_message_id`.
- Статусы `sent` / `pending` / `error` — **локальные**, на провод не выходят; собственное сообщение, пришедшее с сервера (эхо или replay), = `sent`.

Message (wire):

```
{
  "message_id": "m_51c", "seq": 1042, "chat_id": "c_9f2",
  "author_id": "<см. §8.1 – Q11>", "author_label": "Anna",
  "client_message_id": "3f0e...",
  "sent_at": 1755600123,
  "body": {
    "type": "text", "text": "...",
    "attachment": {
      "file_id": "f_77", "name": "report.pdf", "size": 123456,
      "mime": "application/pdf", "expires_at": 1758200000
    }
  }
}
```

- `body` — **единственное место, которое заденет Q1 (граница E2EE)**: при «E2EE на устройстве» объект `body` целиком заменяется на `{ "type": "blob",`

"data": "<base64>"}, и сервер перестаёт его интерпретировать. Реализация уже сейчас обязана не заглядывать внутрь `body` нигде, кроме клиента.

- `Message.seq` — это `seq` события `message.new` данного сообщения: глобальный, внутри одного чата разрезен. `before_seq` = минимальный загруженный `seq` чата; пер-чатовой нумерации не существует — обе реализации держат одну, глобальную.
- `attachment.expires_at` — срок хранения байтов: клиент персистит его вместе с вложением и гасит кнопку Save заранее, а не узнаёт при открытии.
- `client_message_id` присутствует только в собственных сообщениях автора-получателя кадра.

6. События — этап 1 (3 события)

Событие	Когда	Обязанность клиента
<code>message.new</code>	Кто-то написал	Вставка в тред, если открыт; превью и порядок списка клиент обновляет из данных события ; непрочитанное — пересчётом от <code>seq</code> последнего открытия чата (§8.3), инкрементный счётчик недопустим: §3 разрешает дубликаты на стыке <code>replay/live</code>
<code>chat.created</code>	Кто-то создал чат	Строка в списке — включая чат за пределами загруженного префикса (данные события полны; <code>chat.get</code> — для остальных путей)
<code>chat.updated</code>	Переименование	Раскатка по всем поверхностям (механизм <code>WatchChat</code> уже есть). По изменению превью <code>chat.updated</code> не эмитится — превью при новом сообщении обновляет клиент из <code>message.new</code> ; серверный <code>last_message_preview</code> — снимок для страниц <code>chats.list</code>

- События несут **полную** wire-модель сущности, не только `id`.
- **Слияние wire-модели с локальной записью — `merge`, а не `replace`:** устройство-локальные поля (`unread`-состояние чата, `localPath` вложения, локальный статус сообщения) не перезаписываются применением события/эха. Прямое следствие §5 (статусы локальные) и §8.3 (счётчик локальный).
- **Свёртка превью едина** для серверного снимка и клиентского обновления: одна строка, усечение до 120 символов; сообщение-вложение без текста → имя файла. Расхождение формул даст «мигание» строки списка между источниками.
- `ack / error` из инвентаря закрыты конвертом §2 (кадр «ответ»); события этапа 2 — `label.assigned`, `session.expired`, `device.revoked` — §8.1.

7. Файлы — вне сокета, этап 1

Команда (по сокету)	data запроса	Ответ	Дальше (HTTP, §1)
file.uploadBegin	{name, size, mime}	{file_id, upload_url, upload_token, max_attachment_bytes}	PUT байтов с токеном
file.downloadBegin	{file_id}	{download_url, download_token}	GET с поддержкой Range (докачка)

Цепочка вложения: `file.uploadBegin` → PUT байтов → `message.send {attachment: {file_id}}` → сервер собирает объект `attachment` из метаданных загрузки в эхо и в `message.new`. Метаданные (`name` , `size` , `mime`) сервер берёт из `uploadBegin` , а не из байтов; `mime` клиент выводит из расширения имени файла (локальная таблица; неизвестное расширение → `application/octet-stream`) — байты пикер не читает.

Причина выноса: по сокету большой файл блокирует все интерактивные команды (head-of-line), нет докачки, буферизация в память. Токены одноразовые, с TTL.

8. Этап 2 и заблокированное — не реализовывать сейчас

#	Что	Статус / чем заблокировано	Что можно готовить
8.1	Спаривание и аутентификация: <code>pair {token, device_key}</code> (токен любого из четырёх типов), <code>recover {phrase, device_key}</code>, <code>invite.create</code>, <code>device.revoke {device_key}</code> — logout это его частный случай; проверка <code>signature</code> в <code>hello</code>; коды <code>invalid_token</code> / <code>token_expired</code>; <code>identity.setLabel</code>, <code>identity.labelAvailable</code>, <code>identity.completeOnboarding</code>; события <code>label.assigned</code>, <code>session.expired</code>, <code>device.revoked</code>. Протокольный порядок: <code>pair</code> — единственное исключение из правила «первая команда — <code>hello</code>» (неспаренному устройству нечем подписать <code>challenge</code>). Ответ <code>pair</code> / <code>recover</code> обязан различать исходы — «новая личность (онбординг впереди)» vs «устройство добавлено к существующей» — тип токена клиенту не виден по построению <code>payload</code>, а из этого исхода клиент выводит <code>registrationPending</code> / <code>authorized</code>. Logout офлайн: при живой сессии <code>device.revoke</code> уходит до локальной очистки; офлайн — очистка безусловна, осиротевший ключ отзывается с другого устройства (принятая цена). Поверхность <code>Settings</code> «Your ID»/Show QR: источник-секрет исчезает вместе со входом по идентификатору — Show QR становится <code>invite.create</code>, отображение — <code>identity.id</code> из <code>hello</code>	Этап 2. Модель одобрена (authentication.md) и от Q11 не зависит. 🚫 Q11 блокирует только формат <code>author_id / identity.id</code>	Схему таблиц личностей/ устройств/токенов; ловушку Skip решает серверная выдача <code>label</code> в <code>hello</code> (§3)
8.2	<code>push.register</code> — регистрация токена устройства (непрозрачный <code>blob</code>, зашифрованный ключом push-сервера)	🚫 Q2 — открытый пункт «как регистрируется токен и как ключ push-сервера попадает на устройство»; Q13 (контракт с relay)	Поле в схеме БД; сам вызов — после архитектора
8.3	<code>chat.markRead</code>	🟡 Q12 — непрочитанное считает устройство или сервер; спека говорит «устройство», моки везут серверный счётчик	Механика этапа 1 без команды: устройство хранит «<code>seq</code> последнего открытия» на чат; открытый тред продвигает его (вместо нынешней пары

#	Что	Статус / чем заблокировано	Что можно готовить
			инкремент/ <code>markChatRead</code>); бейдж = пересчёт событий выше этой отметки — идемпотентен к дубликатам
8.4	Содержимое <code>body</code> , серверное превью, открытый <code>chat.files</code>	● Q1 — граница E2EE	Конструкция §5 изолирует Q1: конверт, команды и события от него не зависят — §2–§7 реализуемы до решения Q1, замораживать нельзя только формат <code>body</code> (согласовано с документом стека)

9. Требования к реализации соединения

Обе стороны:

- **Keepalive:** ping/pong WebSocket, интервал ~25 с — сотовые NAT сбрасывают простой за ~30 с; полуоткрытый TCP детектируется ask-таймаутом на уровне приложения.
- **Реконнект:** экспоненциальный backoff с джиттером, сброс по успеху; переподключение по событию пробуждения машины, не по таймауту.
- Аутентификация — **одноразовая при установлении соединения**, не на каждый запрос (текущий `AuthInterceptor` с Bearer-токеном — не наша модель).

Фазы сессии на клиенте (второе понятие связности, отдельное от «есть ли интернет»):

нет сокета → connecting → catching-up (replay) → live

Offline-баннер показывают первые две фазы, «догоняем» — индикатор третьей; конец catching-up — по правилу «догнан» из §3. Сегодняшние потребители `ConnectivityService.watchOnline()` , переключаемые на фазы сессии: **три** блока — chats list, thread, chat card; слив очереди исходящих — по входу в `live` .

Переходные требования к клиенту (данные аудита против реального `lib/` ; выполняются до DI-флипа 016, чтобы репозитории и UI при флипе не менялись):

1. **seq** становится ключом хранения, сортировки и пагинации сообщений — вместо сортировки по `sentAt` и математики `page/page_size/total` :
`PageMetadata` / `PagingState` переводятся на `has_more` , «догрузить старше» = `before_seq` от минимального загруженного `seq` чата.
2. **Wire-DTO** переписываются 1:1 под контракт:
`MessageWireEntity` / `ChatWireEntity` эпохи 018 расходятся по именам

(`id` → `message_id`), типам (`sent_at` ISO-строка → `unix-секунды`), структуре (плоский `text` → объект `body` ; вложение → { `file_id`, `name`, `size`, `mime`, `expires_at` }) и несут поля, которых на проводе нет (`status` , `is_system` , `unread_count`) — `required`-поля уронят парсинг контрактного JSON. Категория `FileType` выводится из расширения имени.

3. **Ключ идемпотентности назначается при постановке в очередь** и персистится вместе с ней; повтор после реконнекта и после рестарта идёт с тем же `client_message_id`. Нынешний `local_${counter}` в `ChatThreadBloc` этому не соответствует — шов `sendMessage` расширяется.
4. **Персистентный курсор** `since` на устройство + `seq` в локальной записи каждого сообщения — сегодня поля `seq` нет ни в одной модели, курсор хранить негде.
5. **limits из hello сохраняются в data-слое** и доступны композеру/пикеру для предполётных проверок; `RepositoryException` расширяется кодами провода (сейчас все send-ошибки схлопываются в один `error`).
6. **MessageRemoteDataSource приводится к форме контракта** (без `authorId / authorLabel`; + `client_message_id`, `attachment: {file_id}`) и добавляется файловый датасорс `uploadBegin → PUT → send`.
7. **expires_at добавляется в модели вложения** (домен + Sembast) — гейтинг кнопки Save.
8. **Очередь исходящих** переезжает из `ChatThreadBloc` в data-слой: работает для всех команд, переживает перезапуск приложения.

10. Срез этапа 1 для старта в `client_backend/`

Минимальный срез «пощупать Go», совпадающий с контрактом (ничего не придётся выбрасывать):

1. **Конверт §2 целиком** (включая приветствие с challenge — подпись не проверяется) — ~50 строк на каждой стороне.
2. `session.hello` : `limits` + `identity` + `replay` с `since` и правилом «догнан».
3. `chat.create` , `chats.list` , `message.send` (идемпотентность по `client_message_id`) , `messages.list` с `before_seq` .
4. События `message.new` , `chat.created` через `seq` .
5. SQLite: таблицы `chats` , `messages` , `events` — сообщение и его событие пишутся **одной транзакцией**, broadcast после commit.
6. REST-заготовка §1: `GET /health` сразу; файловые endpoint'ы — следующим шагом (§7).

Референс архитектуры — прототип [docs/client-backend/client_backend_pattern/go-backend](#): его `seq / events /replay ?since=N` /транзакционный outbox — ровно \$3 и \$6

этого контракта. Расхождение одно — REST+push-сокет вместо командного сокета; в `client_backend/` команды сразу идут в конверт §2.